DSLab. 05 Canonical and Standard Forms

Lab. 05 Canonical and Standard Forms

- Design and Verify the following circuits using Verilog HDL
- Schematic
- Verilog HDL
  - Behavioral level modeling
  - Dataflow modeling
  - Structural level modeling
- Please write and upload the lab report (Lab05) -- Due on 2023/09/29 23:59

Operators in Verilog

Table 8.1
Verilog 2001 HDL Operators

Operator Type	Symbol	Operation Performed
Arithmetic	+	addition
	–	subtraction
	*	multiplication
	/	division
	%	modulus
	**	exponentiation
Bitwise or Reduction	~	negation (complement)
	&	AND
	|	OR
	^	exclusive-OR (XOR)
Logical	!	negation
	&&	AND
	||	OR
Shift	>>	logical right shift
	<<	logical left shift
	>>>	arithmetic right shift
	<<<	arithmetic left shift
	{ , }	concatenation
Relational	>	greater than
	<	less than
	==	equality
	!=	inequality
	===	case equality
	!==	case inequality
	>=	greater than or equal
	<=	less than or equal

Copyright ©2012 Pearson Education, publishing as Prentice Hall

Table 8.2
Verilog Operator Precedence

+ – ! ~ & ~& | ~ | ^ ~^ ^~ (unary)	Highest precedence
**	↓
* / %	
+ – (binary)	
<< >> <<< >>>	
< < = > >=	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
| (binary)	
&&	
||	
?: (conditional operator)	
{ } { { } }	Lowest precedence

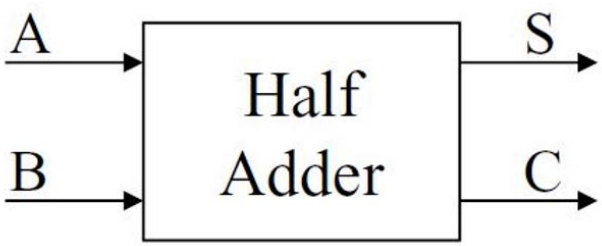
Copyright ©2012 Pearson Education, publishing as Prentice Hall

Use parentheses to enforce your priority

Example 1: Half Adder (Behavioral level modeling)

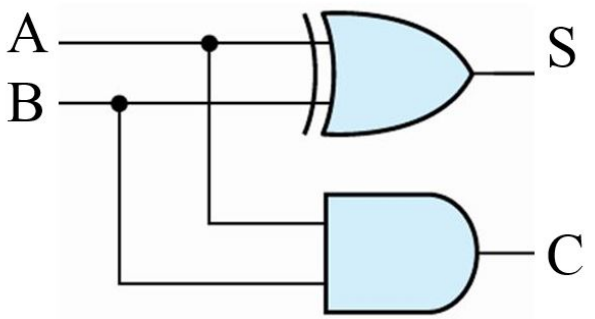
Table 4.3
Half Adder

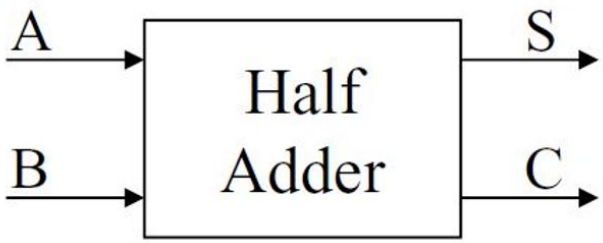
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0




```
module half_adder (S, C, A, B);
    output reg S, C;
    input A, B;

    always @ (A or B)
        {C, S} = A + B;

endmodule
```

Example 2: Half Adder (Dataflow modeling)





$S = A \oplus B$

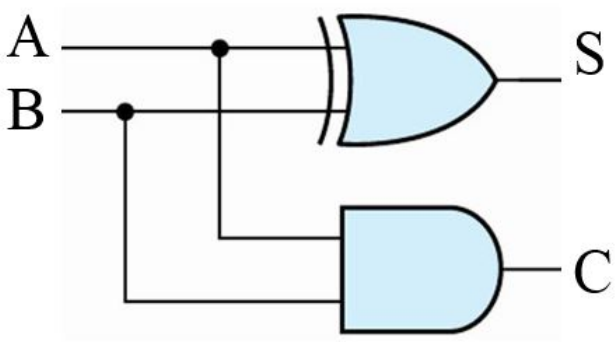
$C = AB$

```
module half_adder (S, C, A, B);
    output S, C;
    input A, B;

    assign S = A ^ B;
    assign C = A & B;

endmodule
```

Verilog Structural Model (Gate Level)

- Built-in gate primitives:
  - **and, nand, nor, or, xor, xnor, buf, not, bufif0, bufif1, notif0, notif1**
- Usage:
  - nand (out, in1, in2); 2-input NAND without delay
  - and #2 (out, in1, in2, in3); 3-input AND with 2 t.u. delay
  - not #1 N1(out, in); NOT with 1 t.u. delay and instance name
  - xor X1(out, in1, in2); 2-input XOR with instance name
- Write them inside module, outside procedures

Example 3: Half Adder (Structural level modeling)

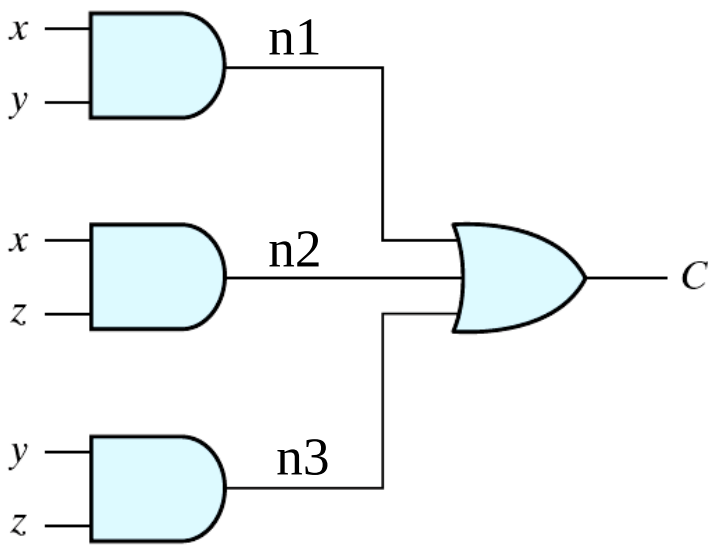


Assuming:

- XOR: 2 t.u. delay
- AND: 1 t.u. delay

```
module half_adder (S, C, A, B);
    output S, C;
    input A, B;

    xor #2 (S, A, B);
    and #1 (C, A, B);

endmodule
```

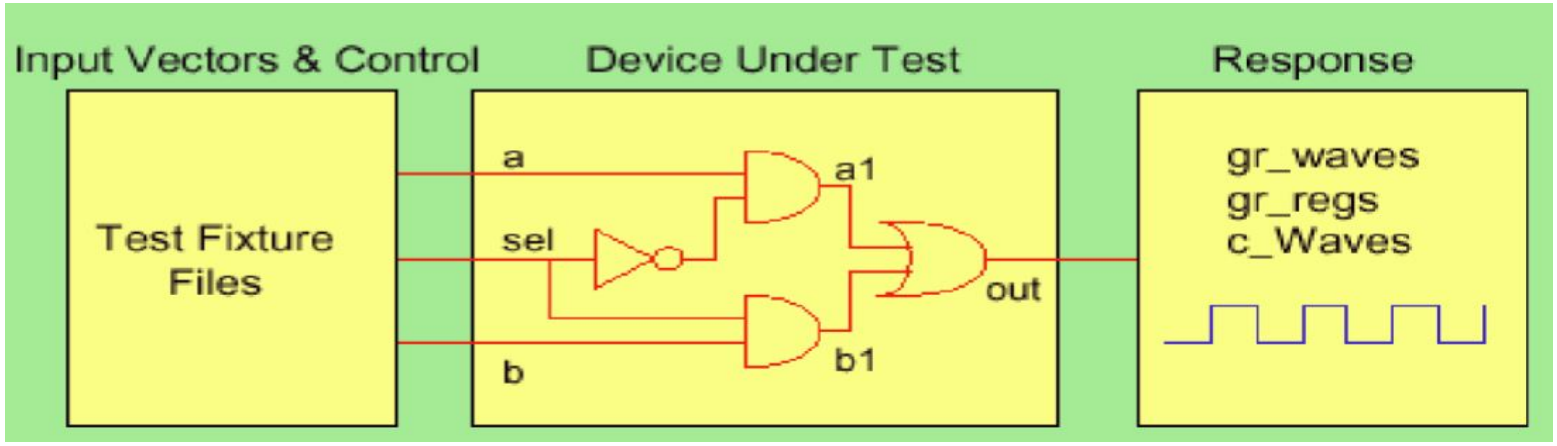
Example 4: Carry of Full Adder (Structural level modeling)

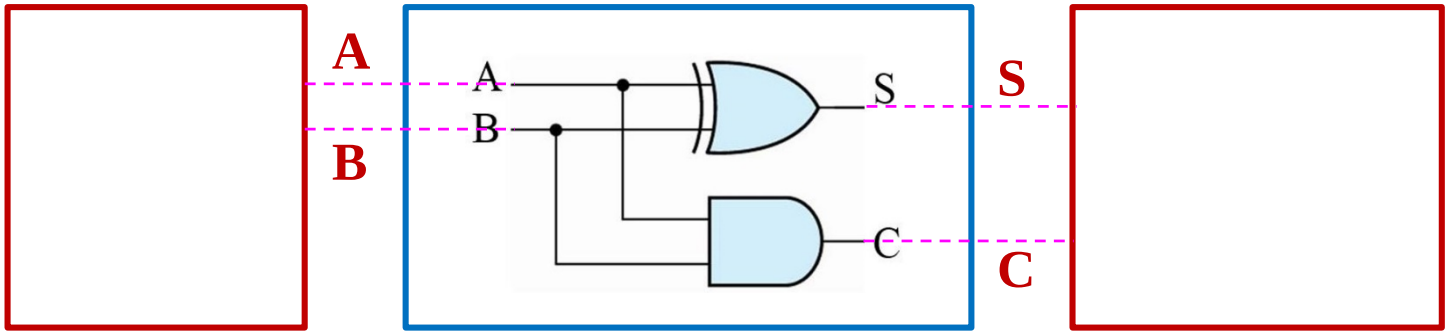


$C = xy + xz + yz$

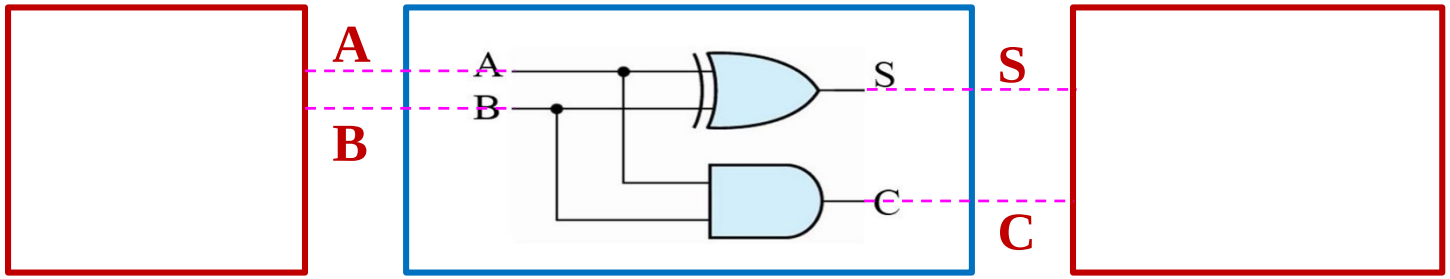
```
module C_FA(C, x, y ,z);
    output C;
    input x, y, z;
    wire n1, n2, n3;

    and (n1, x, y);
    and (n2, x, z);
    and (n3, y, z);
    or  (C, n1, n2, n3);

endmodule
```

Testbench (1/2)





Testbench (2/2)




```
module half_adder (S, C, A, B);
   output S, C;
   input A, B;

   xor #2 (S, A, B);
   and #1 (C, A, B);

endmodule
```

```
`timescale 1ns / 1ps
module t_half_adder();
  reg A;
  reg B;
  wire S;
  wire C;

  half_adder UUT ( .A(A), .B(B), .S(S), .C(C) );

  initial begin
     A=1'b0; B=1'b0;
     #10  A=1'b0; B=1'b1;
     #10  A=1'b1; B=1'b0;
     #10  A=1'b1; B=1'b1;
     #10  $finish;
  end
endmodule
```

Exercise 1

- Find and show the sum of minterms (i.e., *F1*) of the following truth table
- Find and show the product of maxterms (i.e., *F2*) of the following truth table
- Verify *F1* = *F2*

  (using Dataflow modeling)

w	*x*	*y*	*z*	*F*
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Exercise 2

- $F3(x, y, z) = x+z'$
- Find and show the canonical forms of *F3*, i.e., the sum of minterms (*F4*) and the product of maxterms (*F5*)
- Verify $F3 = F4 = F5$

  (using Structural level modeling)

Exercise 3

- *xyz + xyz' + xy'z + x'yz* *(Canonical Form)*

  = xy + xz + yz *(Standard Form)*

  = x(y+z) + yz *(Non-standard Form)*

- Verify *xyz + xyz' + xy'z + x'yz = xy + xz + yz = x(y+z) + yz*
  - using Structural level modeling

$$
\begin{aligned}
F &= xyz + xyz' + xy'z + x'yz \\
&= \underline{xyz} + xyz' + \underline{xyz} + xy'z + \underline{xyz} + x'yz \\
&= xy(z + z') + x(y + y')z + (x + x')yz \\
&= xy + xz + yz
\end{aligned}
$$